I numeri binari relativi

Unità di Apprendimento 3 · Lezione 2

Tecnologie e Progettazione di Sistemi Informatici e di Telecomunicazioni (TPSI)

Classe 3ª · Istituto Tecnico – Informatica

Prof. Dicara

In questa lezione impareremo

- a rappresentare i **numeri relativi** (positivi e negativi) in binario;
- le quattro rappresentazioni: **modulo e segno**, **complemento a 1**, **complemento a 2**, **eccesso 2^{n-1}**;
- che cos'è l'**overflow** con i numeri con segno e come riconoscerlo.

> Problema di partenza: nella memoria ci sono solo 0 e 1. **Come si scrive il segno "−"?**

Mappa della lezione

Parte	Argomento
A	Modulo e segno
B	Complemento a 1
C	Complemento a 2 (la più usata)
D	Overflow nei numeri con segno
E	Rappresentazione in eccesso
F	Confronto e riepilogo

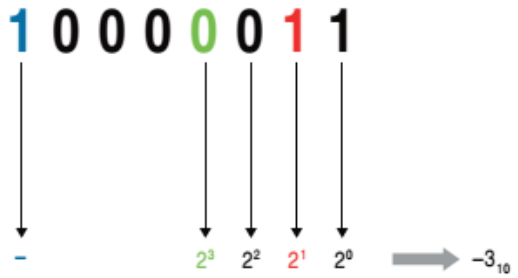
Parte A

Modulo e segno

Modulo e segno: l'idea

> Per rappresentare il segno viene utilizzato il bit a sinistra: per convenzione si associa allo 0 il segno + e a 1 il segno − e si utilizzano i rimanenti bit per codificare il modulo del numero, cioè il valore assoluto del numero.

Il **bit più significativo (MSB)** è il **bit di segno**: **0 = positivo**, **1 = negativo**. Gli altri $n - 1$ bit rappresentano il **modulo** (valore assoluto).



`1000 0011` → segno **1** (negativo), modulo `000 0011` = 3 → **−3**.

Modulo e segno: definizione

MODULO E SEGNO

Nella codifica in **modulo e segno** il bit che rappresenta esplicitamente i segni **+** e **−** (rispettivamente 0 e 1) è l'**MSB** (**Most Significant Bit**); si utilizzano gli altri bit disponibili per rappresentare il valore assoluto come numero.
Il bit MSB prende anche il nome di **bit di segno**.

Esempio (8 bit):

Numero	Segno	Modulo	Codifica
+25	0	`0011001`	`0001 1001`
−25	1	`0011001`	`1001 1001`

Per cambiare segno basta **invertire il bit più significativo**.

Tabella a 3 bit

POSITIVI		NEGATIVI	
000	0	100	−0
001	1	101	−1
010	2	110	−2
011	3	111	−3

Con 3 bit: `000` = +0 e `100` = −0. Esistono **due zeri**: è uno **spreco** e complica i confronti.

Range in modulo e segno

$$[-2^{n-1} + 1, +2^{n-1} - 1]$$

Con *n* bit: da $-(2^{n-1} - 1)$ a $+(2^{n-1} - 1)$.

Bit	Range	Valori distinti
3	−3 … +3	7 ($2^n - 1$)
4	−7 … +7	15
8	−127 … +127	255

Gli 2^n codici disponibili rappresentano solo $2^n - 1$ numeri, perché lo zero è doppio.

Conversioni in modulo e segno

ESEMPI

Convertiamo nei corrispondenti numeri relativi le seguenti rappresentazioni in modulo e segno con n = 5 bit.

- 10101:
  - 1: bit di segno → numero **negativo**;
  - 0101: valore assoluto = 5.

  Il numero relativo corrispondente è quindi $(10101)_2 = (-5)_{10}$
- 01111:
  - 0: bit di segno → numero **positivo**;
  - 1111: valore assoluto = 15.

  Il numero relativo corrispondente è quindi $(01111)_2 = (+15)_{10}$
- 11111:
  - 1: bit di segno → numero **negativo**;
  - 1111: valore assoluto = 15.

  Il numero relativo corrispondente è quindi $(11111)_2 = (-15)_{10}$

Da decimale: segno → bit 0/1; modulo → binario su $n - 1$ bit.

Verso decimale: si calcola il modulo ignorando l'MSB, poi si applica il segno.

Somma in modulo e segno

Non si possono sommare direttamente i bit: **bisogna guardare i segni**.

- **segni concordi** → si sommano i moduli e si mantiene il segno;
- **segni discordi** → si sottrae il modulo minore dal maggiore e si usa il **segno del numero con modulo maggiore**.

		SEGNO	MODULO								
+5	+	0	0	0	0	0	1	0	1	+	
−5	=	1	0	0	0	0	1	0	1	=	
0		1	0	0	0	1	0	1	0		diverso da 0 (!)

Per questo motivo l'hardware deve avere **sommatore e sottrattore** e un circuito di confronto: scomodo.

Parte B

Complemento a 1

Complemento a 1 (CA1)

ESEMPI

- **CA1**(5) → 5 = 00101
- **CA1**(−5) → comp(00101) = 11010
- **CA1**(15) → 15 = 01111
- **CA1**(−15) → comp(01111) = 10000
- **CA1**(0) → 0 = 00000

- **numeri positivi**: come in modulo e segno (MSB = 0);
- **numeri negativi**: si ottengono **invertendo tutti i bit** del corrispondente positivo (MSB = 1).

Esempio (4 bit): +5 = `0101` → −5 = `1010`.

Tabella in complemento a 1

POSITIVI		NEGATIVI	
01111	15	10000	−15
01110	14	10001	−14
…	…	…	…
00010	2	11101	−2
00001	1	11110	−1
00000	0	11111	−0

Osservazioni sul CA1

Osserviamo che:

- i numerali **positivi** iniziano per **0**, i numeri **negativi** per **1**;
- l'intervallo di rappresentazione con n bit è dato da $[-2^{n-1}+1, +2^{n-1}-1]$;
- abbiamo una doppia rappresentazione dello zero.

- anche qui **due zeri**: `0000` (+0) e `1111` (−0);
- range: da $-(2^{n-1}-1)$ a $+(2^{n-1}-1)$, cioè −7 … +7 con 4 bit;
- il **primo bit** indica il segno.

Parte C

Complemento a 2

Complemento a 2 (CA2)

- **positivi**: come in modulo e segno;
- **negativi**: **CA2 del positivo** = inversione dei bit **+ 1**;
- **un solo zero**;
- **range asimmetrico**: da -2^{n-1} a $+(2^{n-1} - 1)$. Con 8 bit: **−128 … +127**.

POSITIVI		NEGATIVI	
01111	15	10000	−16
01110	14	10001	−15
…	…	…	…
00010	2	11101	−3
00001	1	11110	−2
00000	0	11111	−1

Osservazioni sul CA2

Osserviamo che, generalizzando:

- l'intervallo di numeri rappresentati con n bit è $[-2^{n-1}, +2^{n-1} - 1]$;
- nella rappresentazione in complemento a 2 abbiamo l'intervallo asimmetrico con **una sola** rappresentazione dello zero.

Perché il CA2 è lo standard:

1. **un solo zero** (`0000`);
2. **somma e sottrazione** con lo stesso circuito: A − B = A + CA2(B);
3. l'**MSB resta il bit di segno**.

CA2: come calcolare il valore

L'MSB ha **peso negativo -2^{n-1}**; gli altri i pesi positivi.

`1011` (4 bit) = −8 + 0 + 2 + 1 = **−5**

`1111 0011` (8 bit) = −128 + 64 + 32 + 16 + 2 + 1 = **−13**

Codifica di −13 su 8 bit:

1. 13 = `0000 1101`;
2. inverto: `1111 0010`;
3. aggiungo 1: **`1111 0011`**.

Errori tipici con il CA2

Un errore tipico è quello di **dimenticare** il numero di bit con il quale deve essere fatta la rappresentazione.

Primo errore: dimenticare il numero di bit.

ESEMPI

Primo errore

Rappresentazione in **CA2** con 8 bit del numero decimale −3, cioè:

- in binario abbiamo $3_{10} = 11_2$;
- lo rendiamo negativo complementandolo a 2: $-3_{10} = 01_2$, ma otteniamo un risultato positivo!

Procedimento corretto:

Il procedimento corretto per la codifica è quello che segue.

1. Calcoliamo il valore binario su 8 bit di $3_{10} = 00000011_2$.
2. Trasformiamolo in **CA2**: $-3_{10} = 11111101_{CA2}$.

Errori tipici con il CA2 (2)

Secondo errore: complementare anche i numeri positivi.

Secondo errore

Un altro errore comune è quello di **complementare** sempre e comunque anche i numeri positivi.

Per esempio, la giusta codifica del numero +15 è la seguente:

1. calcoliamo il valore binario di $15_{10} = 001111_2$;
2. **fine conversione**, è già codificato in **CA2**.

I numeri **positivi** in CA2 si scrivono **come i binari normali** (con MSB = 0). Si complementano solo i numeri **negativi**.

Parte D

Overflow con i numeri con segno

Overflow nei numeri con segno

Addizione

OVERFLOW

Nelle operazioni aritmetiche in complemento a 2 si definisce **overflow** il traboccamento del riporto sulla cifra più significativa, che in questo tipo di rappresentazione individua il segno del numero (**bit di segno**).

L'**overflow** si verifica quando il risultato **non è rappresentabile** con gli *n* bit disponibili: il risultato ottenuto è **sbagliato**.

Regola pratica (CA2):

- somma di due **positivi** che dà risultato con segno **negativo** → overflow;
- somma di due **negativi** che dà risultato con segno **positivo** → overflow;
- positivo + negativo → **mai** overflow.

Tabella dell'overflow

OPERAZIONE	A	B	BIT DI SEGNO	OVERFLOW
A + B	≥ 0	≥ 0	1	SÌ
A + B	< 0	< 0	0	SÌ
A – B	≥ 0	< 0	1	SÌ
A – B	< 0	≥ 0	0	SÌ

Per ogni riga il bit di segno del risultato è quello "sbagliato" rispetto al segno atteso: è questo che indica l'overflow.

Esempio: 127 + 1 su 8 bit

ESEMPI

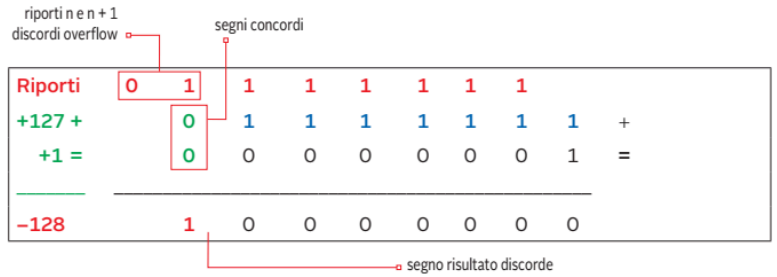
Esempio 1

Il numero positivo più grande rappresentabile con 8 bit in **complemento a 2** è il seguente:

$2^7 - 1 = 127$

cioè la somma di tutti i pesi positivi; sommando 1 a 127 si ottiene −128, che è errato.
Infatti:

$127_{10} = 011111111_{CA2}$



I riporti nelle colonne n e n + 1 sono diversi, quindi siamo in presenza di un **overflow**.

Due positivi hanno prodotto un risultato con bit di segno 1.

Esempio: 100 + 25 e 100 + 50

Esempio 2

Eseguiamo la somma di 100 + 25:



I riporti nelle colonne n e n + 1 sono uguali, quindi il risultato è **corretto**.

Ricorda: **carry ≠ overflow** con i numeri con segno. Un carry può esserci senza overflow (−1 + 1 = `1111` `1111 + 1 = 1 0000 0000` → 0, corretto).

Parte E

Rappresentazione in eccesso

Eccesso 2^{n-1}

Si **somma una costante** (il "bias") al numero relativo, così il risultato è sempre **positivo**, e si codifica in binario.

Quindi per codificare ogni numero si effettuano due passaggi:

- si somma al numero relativo il valore 2^{n-1};
- si codifica in binario il numero così ottenuto.

Due passaggi:

1. si somma al numero relativo il valore 2^{n-1};
2. si codifica in binario il risultato.

Eccesso 8 (4 bit)

-3: 0101, così ottenuto: $(+011)_2 \Rightarrow (1101)_{CA2} \Rightarrow (0101)_{EC8}$ $(= -3 + 8 = 5)$;
$+4$: 1100, così ottenuto: $(-100)_2 \Rightarrow (0100)_{CA2} \Rightarrow (1100)_{EC8}$ $(= +4 + 8 = 12)$;
$+7$: 1001, così ottenuto: $(-111)_2 \Rightarrow (0001)_{CA2} \Rightarrow (1001)_{EC8}$ $(= +9 + 8 = 17)$.

> **Nota di correzione:** in quest'immagine compaiono due errori di stampa. **+7 in eccesso 8** è `7 + 8 = 15 =` **`1111`** (non `1001`/17), e per +4 il numero tra parentesi è **+100**, non −100.

Esempi corretti: −8 → `0000`; −1 → `0111`; 0 → `1000`; +3 → `1011`; +7 → `1111`.

Eccesso 16 (5 bit)

DECIMALE	AGGIUNGIAMO $2^{n-1} = 16$	DECIMALE +16	ECCESSO 16
5	$5 + 2^{n-1} = 5 + 16$	21	10101_{EC16}
−16	$-16 + 16$	0	00000_{EC16}
0	$0 + 16$	16	10000_{EC16}
15	$15 + 16$	31	11111_{EC16}

Tabella completa in eccesso 16

POSITIVI		NEGATIVI	
11111	15	00000	−16
11110	14	00001	−15
…	…	…	…
10010	2	01100	−3
10001	1	01110	−2
10000	0	01111	−1

Nota di correzione: in questa tabella −3 deve valere `-3 + 16 = 13 =` **`01101`** (non `01100`).
Il testo "primi 2N−1" va letto come 2^{n-1} (il bias).

Decodifica dell'eccesso

111 => rappresenta 7 in binario su 3 bit, quindi $2^{n-1} = 2^2 = 4$:
$\text{decod}(111)_{EC4} = 7 - 4 = 3$;

0111 => rappresenta 7 in binario su 4 bit, quindi $2^{n-1} = 2^3 = 8$:
$\text{decod}(0111)_{EC8} = 7 - 8 = -1$;

1011 => rappresenta 11 in binario su 4 bit, quindi $2^{n-1} = 2^3 = 8$:
$\text{decod}(1011)_{EC8} = 11 - 8 = 3$;

01011 => rappresenta 11 in binario su 5 bit, quindi $2^{n-1} = 2^4 = 16$:
$\text{decod}(01011)_{EC16} = 11 - 16 = -5$.

Procedura: si legge il numero come binario normale e **si sottrae 2^{n-1}**.

> **Curiosità:** l'eccesso coincide con il CA2 **con il bit di segno invertito**. Si usa per **l'esponente dei numeri in virgola mobile** (Lezione 3).

Confronto delle rappresentazioni (4 bit)

Dec	Modulo e segno	CA1	CA2	Eccesso 8
−8	—	—	`1000`	`0000`
−7	`1111`	`1000`	`1001`	`0001`
−5	`1101`	`1010`	`1011`	`0011`
−3	`1011`	`1100`	`1101`	`0101`
−1	`1001`	`1110`	`1111`	`0111`
+0	`0000`	`0000`	`0000`	`1000`
+3	`0011`	`0011`	`0011`	`1011`
+7	`0111`	`0111`	`0111`	`1111`

(in M&S e CA1 esiste anche −0: `1000` e `1111` rispettivamente)

Riepilogo: i punti chiave

	Zeri	Range (n bit)	Uso
Modulo e segno	2	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	didattico
CA1	2	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	didattico
CA2	**1**	**-2^{n-1} … $+(2^{n-1}-1)$**	**interi nei computer**
Eccesso 2^{n-1}	1	-2^{n-1} … $+(2^{n-1}-1)$	esponente dei float

- Sempre indicare **quanti bit** si usano.
- **Overflow** se il segno del risultato è incoerente con quello degli addendi.

Verifica: esercizi

1. Scrivi su 8 bit in **modulo e segno**, **CA1** e **CA2** il numero −45.
2. Quanto vale `1110 0110` in CA2? E in modulo e segno?
3. Qual è il range in CA2 con 6 bit? E in modulo e segno?
4. Calcola in CA2 su 8 bit: `50 + 90`. C'è overflow?
5. Calcola in CA2 su 8 bit: `30 − 50`.
6. Codifica in **eccesso 8** (4 bit) i numeri −6, 0 e +5.
7. Decodifica `10110` in **eccesso 16**.
8. Perché due zeri sono un problema?

Verifica: soluzioni

1. 45 = `0010 1101`. **M&S:** `1010 1101`; **CA1:** `1101 0010`; **CA2:** `1101 0011`.
2. CA2: −128 + 64 + 32 + 4 + 2 = **−26**. M&S: segno −, modulo `110 0110` = 102 → **−102**.
3. CA2 6 bit: **−32 … +31**; M&S: **−31 … +31**.
4. 50 + 90 = 140 > 127 → `0011 0010 + 0101 1010 = 1000 1100` = −116 → **overflow**.
5. 30 − 50 = 30 + CA2(50) = `0001 1110 + 1100 1110 = 1110 1100` = **−20** ✓ (nessun overflow).
6. −6 → −6 + 8 = 2 → **`0010`**; 0 → **`1000`**; +5 → 13 → **`1101`**.
7. `10110` = 22 → 22 − 16 = **+6**.
8. Spreco di un codice, e i confronti (`+0 == -0`?) richiedono controlli aggiuntivi.

Grazie per l'attenzione

Domande?